

```

1. #include <stdio.h>
2. #include <conio.h>
3. void main(){
4. int i=1;//initializing a local variable
5. clrscr();
6. //starting a loop from 1 to 10
7. for(i=1;i<=10;i++){
8. if(i==5){//if value of i is equal to 5, it will continue the loop
9. continue;
10. }
11. printf("%d \n",i);
12. }//end of for loop
13. getch();
14. }

```

Output

1234678910

Comparision between break and continue statements

Break	Continue
1 : break statement takes the control to the outside of the loop	1 :continue statement takes the control to the beginning of the loop..
2 : it is also used in switch statement.	2 : This can be used only in loop statements.
3 : Always associated with if condition in loops.	3 : This is also associated with if condition.

ARRAYS

Using Arrays in C

C supports a derived data type known as *array* that can be used to handle large amounts of data (multiple values) at a time.

Definition:

An array is a group (or collection) of same data types.

Or

An array is a collection of data that holds fixed number of values of same type.

Or

Array is a *collection* or *group* of elements (data). All the elements of array are *homogeneous* (similar). **It has contiguous memory location.**

Or

An array is a data structured that can store a fixed size sequential collection of elements of same data type.

What's the need of an array?

Suppose you have to store marks of 50 students, one way to do this is allotting 50 variables. So it will be typical and hard to manage. For example we can not access the value of these variables with only 1 or 2 lines of code.

Another way to do this is array. By using array, we can access the elements easily. Only few lines of code is required to access the elements of array.

Where arrays are used

- to store list of Employee or Student names,
- to store marks of a students,
- or to store list of numbers or characters etc.

Advantage of C Array

1) Code Optimization: Less code to the access the data.

2) Easy to traverse data: By using the for loop, we can retrieve the elements of an array easily.

3) Easy to sort data: To sort the elements of array, we need a few lines of code only.

4) Random Access: We can access any element randomly using the array.

Disadvantage of Array

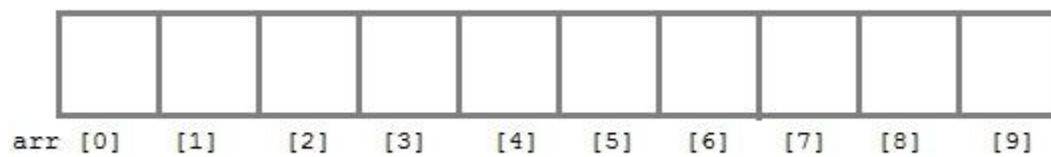
Fixed Size: Whatever size, we define at the time of declaration of array, we can't exceed the limit. So, it doesn't grow the size dynamically like LinkedList

Declaration of an Array

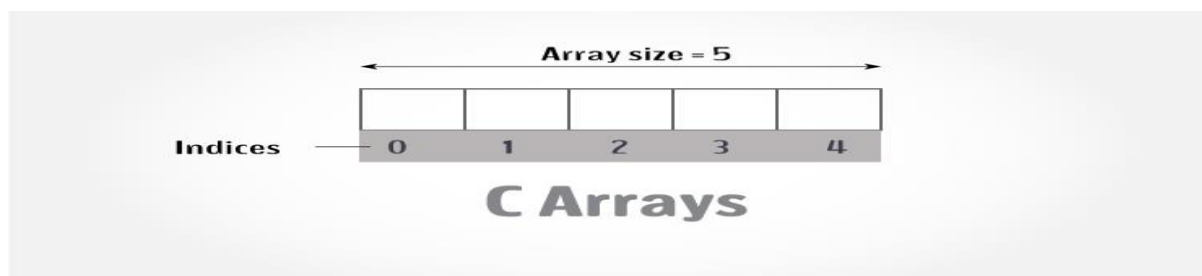
data-type variable-name[size/length of array];

For example:

```
int arr[10];
```



```
int arr[ 5];
```



Here **int** is the data type, **arr** is the name of the array and 10 is the size of array. It means array **arr** can only contain 10 elements of **int** type. **Index** of an array starts from 0 to size-1 i.e first element of **arr** array will be stored at arr[0] address and last element will occupy arr[9].

Initialization of an Array

After an array is declared it must be initialized. Otherwise, it will contain **garbage** value(any random value). An array can be initialized at either **compile time** or at **runtime**.

Compile time Array initialization

Compile time initialization of array elements is same as ordinary variable initialization.

Syntax : `data_type array_name[size]={v1,v2,...vn/list of values ;`

Example

```
int age[5]={22,25,30,32,35};
```

	0	1	2	3	4
age	22	25	30	32	35

```
int marks[4]={ 67, 87, 56, 77 }; //integer array initialization
```

```
float area[5]={ 23.4, 6.8, 5.5 }; //float array initialization
```

```
int marks[4]={ 67, 87, 56, 77, 59 }; //Compile time error
```

Different ways of initializing arrays :

- 1 : Initializing all specified memory locations
- 2 : Partial array initialization.
- 3 : Initialization without size.
- 4 : String initialization.

1 : Initializing all specified memory locations : If the number of values to be initialized is equal to size of array. Arrays can be initialized at the time of declaration. Array elements can be initialized with data items of type int,float,char, etc.

Ex : consider integer initialization

```
int a[5]={10,20,30,40,50};
```

During compilation, 5 contiguous memory locations are reserved by the compiler for the variable a and all these locations are initialized.

The array a is initialized as

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	30	40	50
1000	1002	1004	1006	1008

If the size of integer is 2 bytes, 10 bytes will be allocated for the variable a.

Ex : consider character initialization

```
char b[8] = {'C','O','M','P','U','T','E','R'};
```

The array b is initialized as

b[0]	b[1]	b[2]	b[3]	b[4]	b[5]	b[6]	b[7]
C	O	M	P	U	T	E	R

Other Examples : `char b[5]={ 'J','B','R','E','C','B'};`

`//error : number of initial values are more than the size of array.`

Other Example : `int a[5]={10,20,30,40,50,60};`

//error : Number of initial values are more than the size of array.

2 : Partial Array Initialization : partial array initialization is possible in C language. If the number of values to be initialized is less than the size of the array, then the elements are initialized in the order from 0th location. The remaining locations will be initialized to zero automatically.

Ex : Consider the partial initialization

```
int a[5]={10,15};
```

Eventhough compiler allocates 5 memory locations, using this declaration statement, the compiler initializes first two locations with 10 and 15, the next set of memory locations are automatically initialized to zero.

The array a is partial initialization as

a[0]	a[1]	a[2]	a[3]	a[4]
10	15	0	0	0
1000	1002	1004	1006	1008

How to access the elements of an array?

You can access elements of an array by **indices/index**. You can use array subscript (or index) to access any element stored in array. Subscript starts with 0, which means array_name[0] would be used to access first element in an array.

In general array_name[n-1] can be used to access nth element of an array. where n is any integer number.

Example

```
float mark[5];
```

Suppose you declared an array mark as above. The first element is mark[0], second element is mark[1] and so on.

mark[0] mark[1] mark[2] mark[3] mark[4]

--	--	--	--	--

Few key notes:

- Arrays have 0 as the first index not 1. In this example, mark[0]
- If the size of an array is n, to access the last element, (n-1) index is used. In this example, mark[4]
- Suppose the starting address of mark[0] is 2120d. Then, the next address, a[1], will be 2124d, address of a[2] will be 2128d and so on. It's because the size of a float is 4 bytes.

Input data into array

As you can see, in above example that I have used 'for loop' and 'scanf statement' to enter data into array. You can use any loop for data input.

Code:

```
for (x=0; x<=19;x++)  
{  
    printf("enter the integer number %d\n", x);  
    scanf("%d", &num[x]);  
}
```

Reading out data from an array

For example you want to read and display array elements, you can do it just by using any loop. Suppose array is mydata[20].

```
for (int i=0; i<20; i++)  
{  
    printf("%d\n", mydata[i]);  
}
```

Exmaple

```
#include<stdio.h>

#include<conio.h>

void main()

{

    int i;

    int arr[]={2,3,4};  //Compile time array initialization

    for(i=0 ; i<3 ; i++) {

        printf("%dt",arr[i]);

    }

    getch();

}
```

Output

2 3 4

Exmaple

1. include <stdio.h>
2. #include <conio.h>
3. **void** main(){
4. **int** i=0;
5. **int** marks[5]={20,30,40,50,60};*//declaration and initialization of array*
6. clrscr();
- 7.
8. *//traversal of array*
9. **for**(i=0;i<5;i++){
10. printf("%d \n",marks[i]);
11. }
- 12.
13. getch();
14. }

Output

20
30
40
50
60

Runtime Array initialization

An array can also be initialized at runtime using scanf() function. This approach is usually used for initializing large array, or to initialize array with user specified values.

Example

```
#include<stdio.h>

#include<conio.h>

void main()

{

    int arr[4];

    int i, j;

    printf("Enter array element");

    for(i=0;i<4;i++)

    {

        scanf("%d",&arr[i]);  //Run time array initialization

    }

    for(j=0;j<4;j++)

    {

        printf("%d\n",arr[j]);

    }

    getch();
```

}

Two-Dimensional Arrays

The two dimensional array in C language is represented in the form of rows and columns, also known as matrix. It is also known as *array of arrays* or *list of arrays*.

The two dimensional, three dimensional or other dimensional arrays are also known as *multidimensional* arrays.

Declaration of two dimensional Array

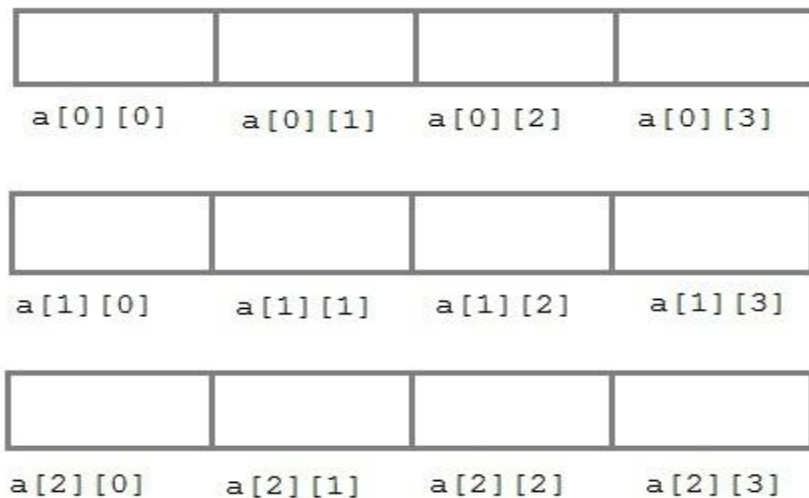
```
data_type array_name[size1][size2];
```

Example

```
int twodimen[4][3];
```

Example :

```
int a[3][4];
```



Initialization of 2D Array

```
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
```

Accessing Two-Dimensional Array Elements

An element in a two-dimensional array is accessed by using the subscripts, i.e., row index and column index of the array.

Example

```
1. #include <stdio.h>
2. #include <conio.h>
3. void main(){
4. int i=0,j=0;
5. int arr[4][3]={ {1,2,3},{2,3,4},{3,4,5},{4,5,6} };
6. clrscr();
7. //traversing 2D array
8. for(i=0;i<4;i++){
9.   for(j=0;j<3;j++){
10.    printf("arr[%d] [%d] = %d \n",i,j,arr[i][j]);
11.   }//end of j
12. }//end of i
13. getch();
14. }
```

Output

```
arr[0][0] = 1
arr[0][1] = 2
arr[0][2] = 3
arr[1][0] = 2
arr[1][1] = 3
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 4
arr[2][2] = 5
arr[3][0] = 4
arr[3][1] = 5
arr[3][2] = 6
```

Example Write a C program Addition of Two Matrices

```
#include<stdio.h>
```

```
#include<conio.h>
```

```

void main()

{

    int a[25][25],b[25][25],c[25][25],i,j,m,n;

    clrscr();

    printf("enter the rows and columns of two matrices:\n");

    scanf("%d%d",&m,&n);

    printf("\nenter the elements of A matrices");

    for(i=0;i<m;i++)

    {

        for(j=0;j<n;j++)

            scanf("\t%d",&a[i][j]);

    }

    printf("\nenter the elements of B matrices");

    for(i=0;i<m;i++)

    {

        for(j=0;j<n;j++)

            scanf("\t%d",&b[i][j]);

    }

    printf("\nThe elements of A matrices");

    for(i=0;i<m;i++)

    {

        printf("\n");

        for(j=0;j<n;j++)

```

```
        printf("\t%d",a[i][j]);  
    }  
    printf("\n\nThe elements of B matrices");  
    for(i=0;i<m;i++)  
    {  
        printf("\n");  
        for(j=0;j<n;j++)  
            printf("\t%d",a[i][j]);  
    }  
    printf("\n\nThe additon of two matrices");  
    for(i=0;i<m;i++)  
    {  
        printf("\n");  
        for(j=0;j<n;j++)  
        {  
            c[i][j]=a[i][j]+b[i][j];  
            printf("\t%d",c[i][j]);  
        }  
    }  
    getch();  
}
```

Write a C program Multiplication of Two Matrices.

```
#include<stdio.h>

#include<conio.h>

void main()

{

    int a[25][25],b[25][25],c[25][25],i,j,m,n,k,r,s;

    clrscr();

    printf("enter the rows and columns of A matrices:\n");

    scanf("%d%d",&m,&n);

    printf("enter the rows and columns of B matrices:\n");

    scanf("%d%d",&r,&s);

    printf("\nenter the elements of A matrices");

    for(i=0;i<m;i++)

    {

        for(j=0;j<n;j++)

            scanf("\t%d",&a[i][j]);

    }

    printf("\nenter the elements of B matrices");

    for(i=0;i<m;i++)

    {

        for(j=0;j<n;j++)

            scanf("\t%d",&b[i][j]);
```

```

}

printf("\nThe elements of A matrices");

for(i=0;i<m;i++)

{

    printf("\n");

    for(j=0;j<n;j++)

        printf("\t%d",a[i][j]);

}

printf("\nThe elements of B matrices");

for(i=0;i<m;i++)

{

    printf("\n");

    for(j=0;j<n;j++)

        printf("\t%d",b[i][j]);

}


for(i=0;i<m;i++)

{

    printf("\n");

    for(j=0;j<n;j++)

    {

        c[i][j]=0;

        for(k=0;k<m;k++)

```

```

        c[i][j]=c[i][j]+a[i][k]*b[k][j];

    }

}

printf("\nThe Multiplication of two matrices");

for(i=0;i<m;i++)

{

    printf("\n");

    for(j=0;j<n;j++)

        printf("\t%d",c[i][j]);

    }

    getch();

}

```

Multidimensional Arrays

How to initialize a multidimensional array?

Initialization of a three dimensional array.

You can initialize a three dimensional array in a similar way like a two dimensional array. Here's an example

```

int test[2][3][4] = {

    { {3, 4, 2, 3}, {0, -3, 9, 11}, {23, 12, 23, 2} },

    { {13, 4, 56, 3}, {5, 9, 3, 5}, {3, 1, 4, 9} }

};

```

Example

```

#include <stdio.h>

int main()
{
    // this array can store 12 elements
    int i, j, k, test[2][3][2];
    printf("Enter 12 values: \n");
    for(i = 0; i < 2; ++i) {
        for (j = 0; j < 3; ++j) {
            for(k = 0; k < 2; ++k ) {
                scanf("%d", &test[i][j][k]);
            }
        }
    }

    // Displaying values with proper index.
    printf("\nDisplaying values:\n");
    for(i = 0; i < 2; ++i) {
        for (j = 0; j < 3; ++j) {
            for(k = 0; k < 2; ++k ) {
                printf("test[%d][%d][%d] = %d\n", i, j, k, test[i][j][k]);
            }
        }
    }

    return 0;
}

```

Output

Enter 12 values:

123456789101112

Displaying Values:

test[0][0][0] = 1

test[0][0][1] = 2

test[0][1][0] = 3

test[0][1][1] = 4

test[0][2][0] = 5

test[0][2][1] = 6

test[1][0][0] = 7

test[1][0][1] = 8

test[1][1][0] = 9

test[1][1][1] = 10

test[1][2][0] = 11

test[1][2][1] = 12

STRINGS:

String Concepts

String is an *array of characters* that is terminated by \0 (null character). This null character indicates the end of the string. Strings are always enclosed by double quotes (" "). Whereas, character is enclosed by single quotes.

Or